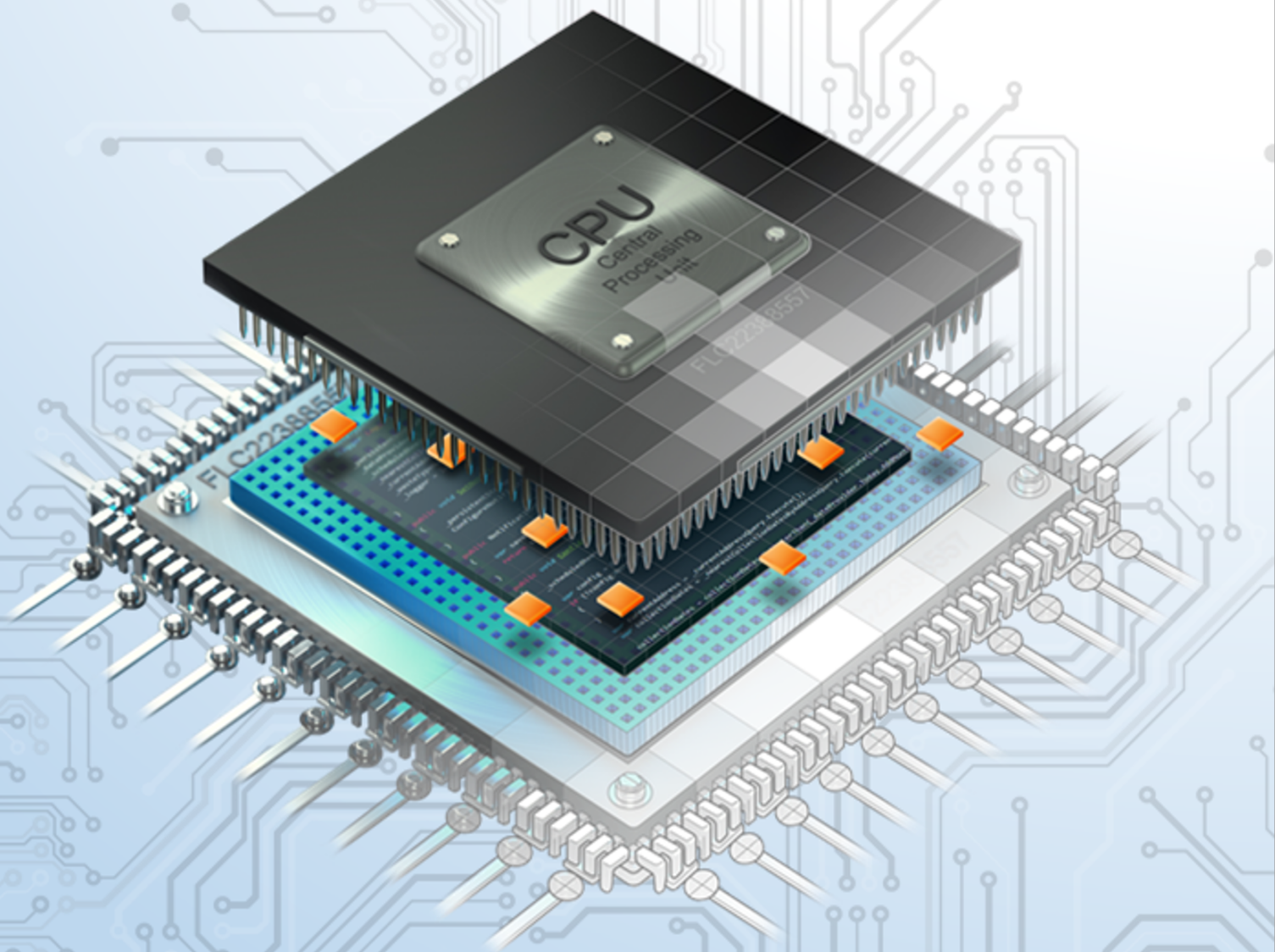




HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller

>_@Nguyễn_Hưng_Thịnh\2313287
>_cd REPORT_LAB_2



Instructor: Mr_Tôn_Huỳnh_Long

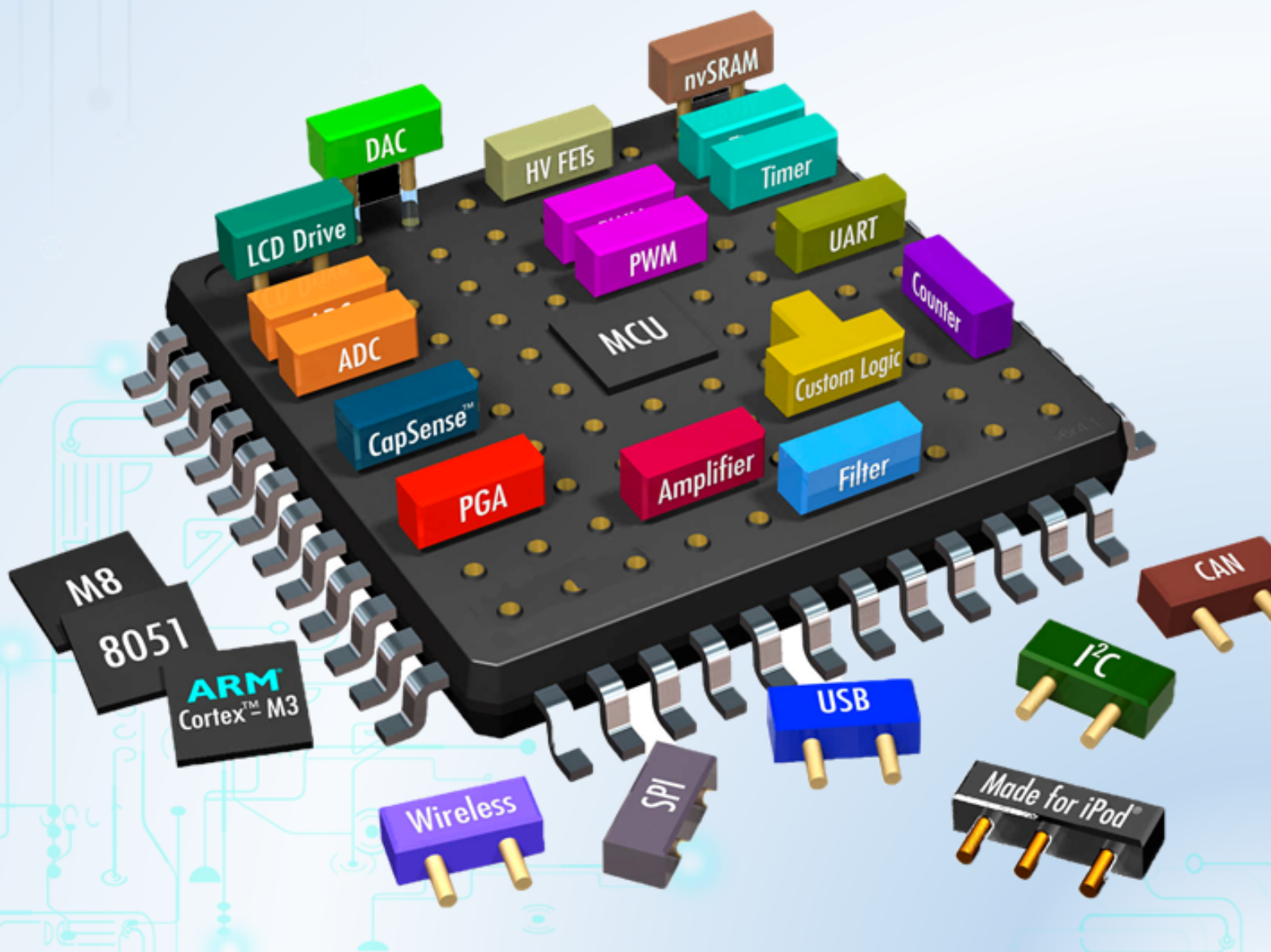
Dr. Le Trong Nhan

Mục lục

Chapter 1. Timer Interrupt and LED Scanning	5
1 Introduction	6
2 Timer Interrupt Setup	8
3 Exercise and Report	11
3.1 Exercise 1	11
3.2 Exercise 2	13
3.3 Exercise 3	16
3.4 Exercise 4	19
3.5 Exercise 5	20
3.6 Exercise 6	21
3.7 Exercise 7	23
3.8 Exercise 8	23
3.9 Exercise 9	25
3.10 Exercise 10	28

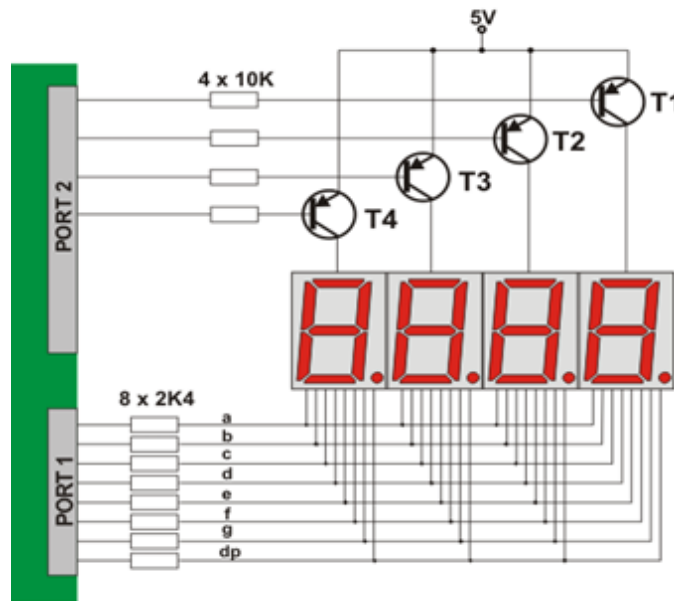
CHƯƠNG 1

Timer Interrupt and LED Scanning



1 Introduction

Timers are one of the most important features in modern micro-controllers. They allow us to measure how long something takes to execute, create non-blocking code, precisely control pin timing, and even run operating systems. In this manual, how to configure a timer using STM32CubeIDE is presented how to use them to flash an LED. Finally, students are proposed to finalize 10 exercises using timer interrupt for applications based LED Scanning.



Hình 1.1: Four seven segment LED interface for a micro-controller

Design an interface for with multiple LED (seven segment or matrix) displays which is to be controlled is depends on the number of input and output pins needed for controlling all the LEDs in the given matrix display, the amount of current that each pin can source and sink and the speed at which the micro-controller can send out control signals. With all these specifications, interfacing can be done for 4 seven segment LEDs with a micro-controller is proposed in the figure above.

In the above diagram each seven segment display is having 8 internal LEDs, leading to the total number of LEDs is 32. However, not all the LEDs are required to turn ON, but one of them is needed. Therefore, only 12 lines are needed to control the whole 4 seven segment LEDs. By controlling with the micro-controller, we can turn ON an LED during a same interval T_s . Therefore, the period for controlling all 4 seven segment LEDs is $4T_s$. In other words, these LEDs are scanned at frequency $f = 1/4T_s$. Finally, it is obviously that if the frequency is greater than 30Hz (e.g. $f = 50\text{Hz}$), it seems that all LEDs are turn ON at the same time.

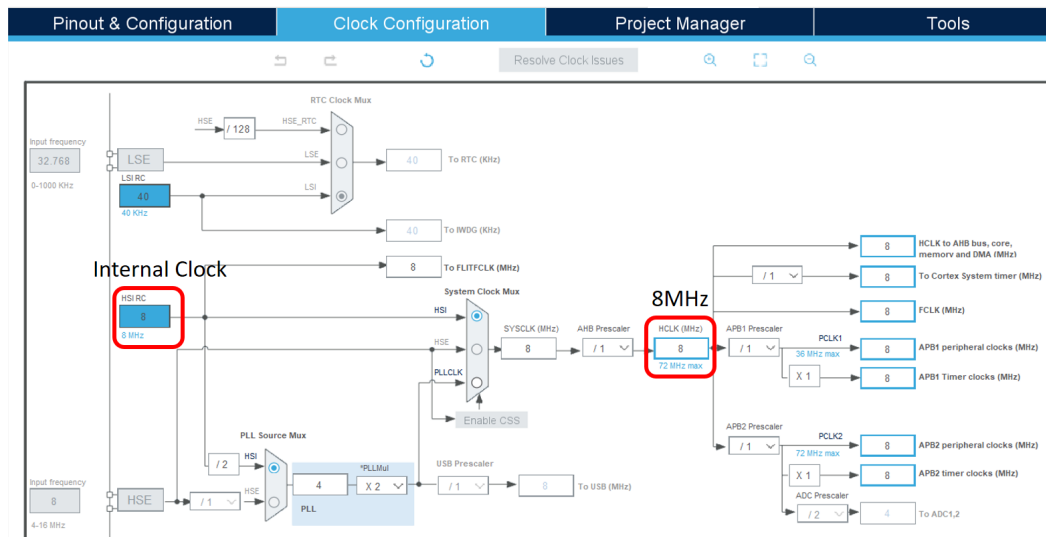
In this manual, the timer interrupt is used to design the interval T_s for LED scanning. Unfortunately, the simulation on Proteus can not execute at high frequency, the frequency f is set to a low value (e.g. 1Hz). In a real implementation, this fre-

quency should be 50Hz.

2 Timer Interrupt Setup

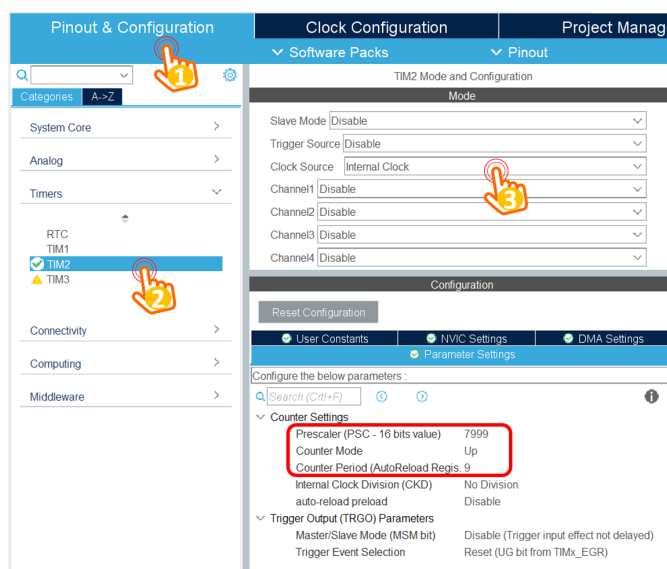
Step 1: Create a simple project, which LED connected to PA5. The manual can be found in the first lab.

Step 2: Check the clock source of the system on the tab **Clock Configuration** (from *.ioc file). In the default configuration, the internal clock source is used with 8MHz, as shown in the figure bellow.



Hình 1.2: Default clock source for the system

Step 3: Configure the timer on the **Parameter Settings**, as follows:

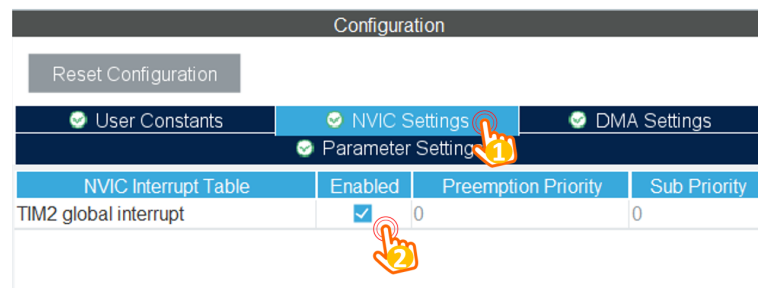


Hình 1.3: Configure for Timer 2

Select the clock source for timer 2 to the **Internal Clock**. Finally, set the prescaler and the counter to 7999 and 9, respectively. These values are explained as follows:

- The target is to set an interrupt timer to 10ms
- The clock source is 8MHz, by setting the prescaler to 7999, the input clock source to the timer is $8\text{MHz}/(7999+1) = 1000\text{Hz}$.
- The interrupt is raised when the timer counter is counted from 0 to 9, meaning that the frequency is divided by 10, which is 100Hz.
- The frequency of the timer interrupt is 100Hz, meaning that the period is $1/100\text{Hz} = 10\text{ms}$.

Step 4: Enable the timer interrupt by switching to **NVIC Settings** tab, as follows:



Hình 1.4: Enable timer interrupt

Finally, save the configuration file to generate the source code.

Step 5: On the **main()** function, call the timer init function, as follows:

```

1 int main(void)
2 {
3     HAL_Init();
4     SystemClock_Config();
5
6     MX_GPIO_Init();
7     MX_TIM2_Init();
8
9     /* USER CODE BEGIN 2 */
10    HAL_TIM_Base_Start_IT(&htim2);
11    /* USER CODE END 2 */
12
13    while (1){
14
15    }
16 }
```

Program 1.1: Init the timer interrupt in main

Please put the init function in a right place to avoid conflicts when code generation is executed (e.g. ioc file is updated).

Step 6: Add the interrupt service routine function, this function is invoked every 10ms, as follows:

```
1 /* USER CODE BEGIN 4 */
2 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
3 {
4 }
5 /* USER CODE END 4 */
```

Program 1.2: Add an interrupt service routine

Step 7: To run a LED Blinky demo using interrupt, a short manual is presented as follows:

```
1 /* USER CODE BEGIN 4 */
2 int counter = 100;
3 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
4 {
5     counter--;
6     if(counter <= 0){
7         counter = 100;
8         HAL_GPIO_TogglePin(LED_RED_GPIO_Port , LED_RED_Pin);
9     }
10 }
/* USER CODE END 4 */
```

Program 1.3: LED Blinky using timer interrupt

The **HAL_TIM_PeriodElapsedCallback** function is an infinite loop, which is invoked every cycle of the timer 2, in this case, is 10ms.

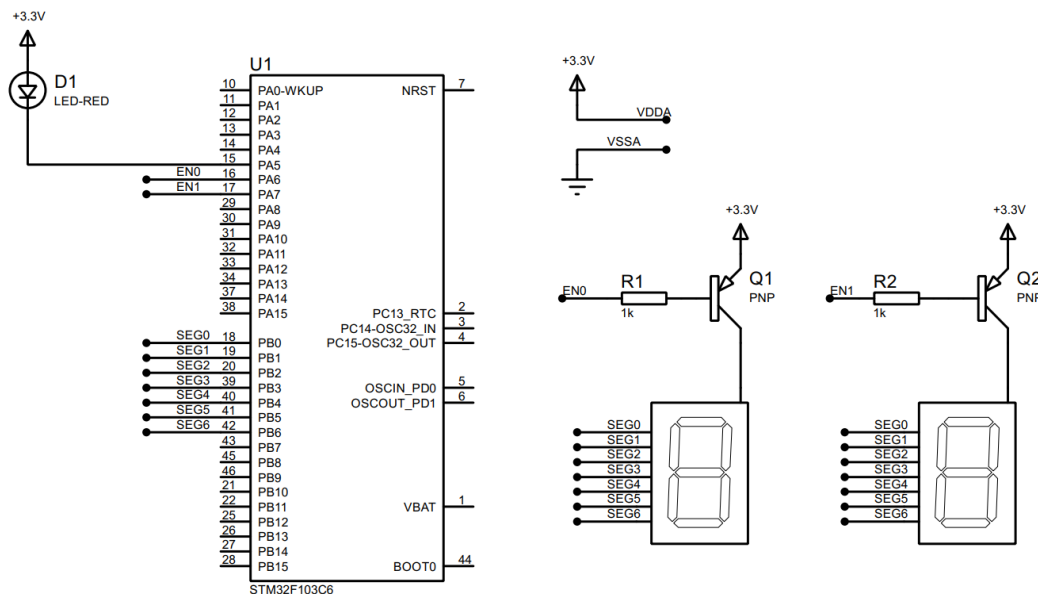
3 Exercise and Report

3.1 Exercise 1

The first exercise show how to interface for multiple seven segment LEDs to STM32F103C6 micro-controller (MCU). Seven segment displays are common anode type, meaning that the anode of all LEDs are tied together as a single terminal and cathodes are left alone as individual pins.

In order to save the resource of the MCU, individual cathode pins from all the seven segment LEDs are connected together, and connect to 7 pins of the MCU. These pins are popular known as the **signal pins**. Meanwhile, the anode pin of each seven segment LEDs are controlled under a power enabling circuit, for instance, an PNP transistor. At a given time, only one seven segment LED is turned on. However, if the delay is small enough, it seems that all LEDs are enabling.

Implement the circuit simulation in Proteus with two 7-SEGMENT LEDs as following:



Hình 1.5: Simulation schematic in Proteus

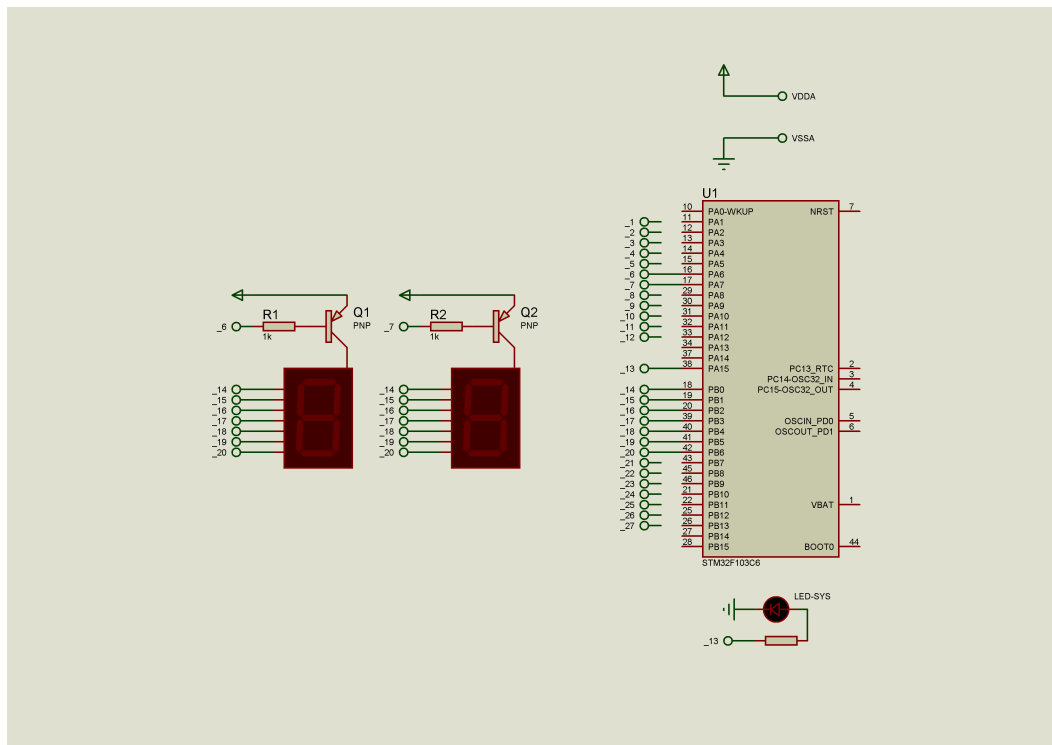
Components used in the schematic are listed bellow:

- 7SEG-COM-ANODE (connected from PB0 to PB6)
- LED-RED
- PNP
- RES
- STM32F103C6

Students are proposed to use the function **display7SEG(int num)** in the Lab 1 in this exercise. Implement the source code in the interrupt callback function to display number "1" on the first seven segment and number "2" for second one. The switching time between 2 LEDs is half of second.

Report 1: Capture your schematic from Proteus and show in the report.

My schematic:



Hình 1.6: Simulation schematic for Lab2 Ex1 in Proteus.

Report 2: Present your source code in the **HAL_TIM_PeriodElapsedCallback** function.

My code:

```
1 int timerLED_SYS = 50;
2 int timer1 = 10;
3 int index_led = 0;
4 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
5 {
6     timerLED_SYS--;
7     timer1--;
8     if (timerLED_SYS <= 0){
9         timerLED_SYS = 50;
10        HAL_GPIO_TogglePin(LED_SYS_GPIO_Port , LED_SYS_Pin);
11    }
12    if (timer1 <= 0){
13        timer1 = 10;
14    }
15 }
```

```

13     if (index_led == 0){
14         display7SEG(1);
15         HAL_GPIO_WritePin(PA6_GPIO_Port , PA6_Pin ,
GPIO_PIN_RESET);
16         HAL_GPIO_WritePin(PA7_GPIO_Port , PA7_Pin ,
GPIO_PIN_SET);
17         index_led = 1;
18     }
19     else if (index_led == 1){
20         display7SEG(2);
21         HAL_GPIO_WritePin(PA6_GPIO_Port , PA6_Pin ,
GPIO_PIN_SET);
22         HAL_GPIO_WritePin(PA7_GPIO_Port , PA7_Pin ,
GPIO_PIN_RESET);
23         index_led = 0;
24     }
25 }
26 }

```

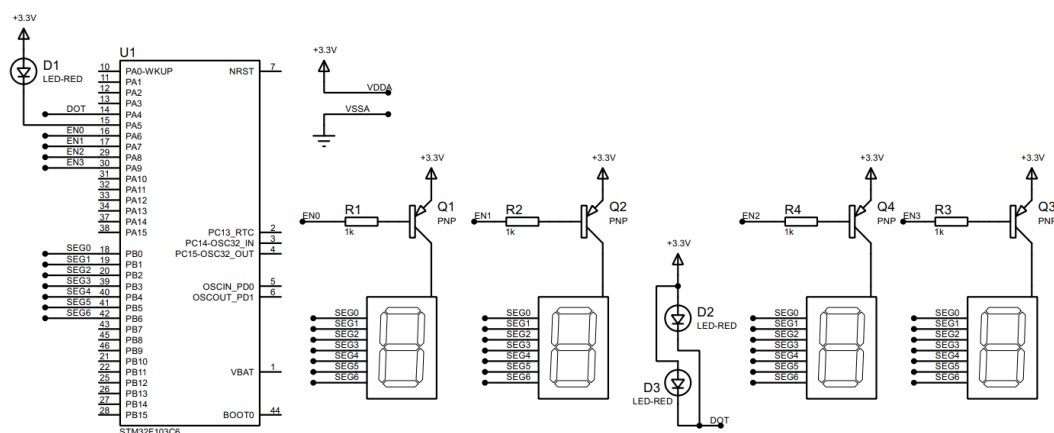
Program 1.4: My code for Lab2 Ex1

Short question: What is the frequency of the scanning process?

My answer: The frequency of the scanning process is 5Hz. As the switching time between 2 SEGMENTS is by 10 unit (= 100ms in this configuration), the period for scanning 2 SEGMENTS is 20 unit (= 200ms). Therefore, the frequency is $1/0.2s = 5Hz$.

3.2 Exercise 2

Extend to 4 seven segment LEDs and two LEDs (connected to PA4, labeled as **DOT**) in the middle as following:



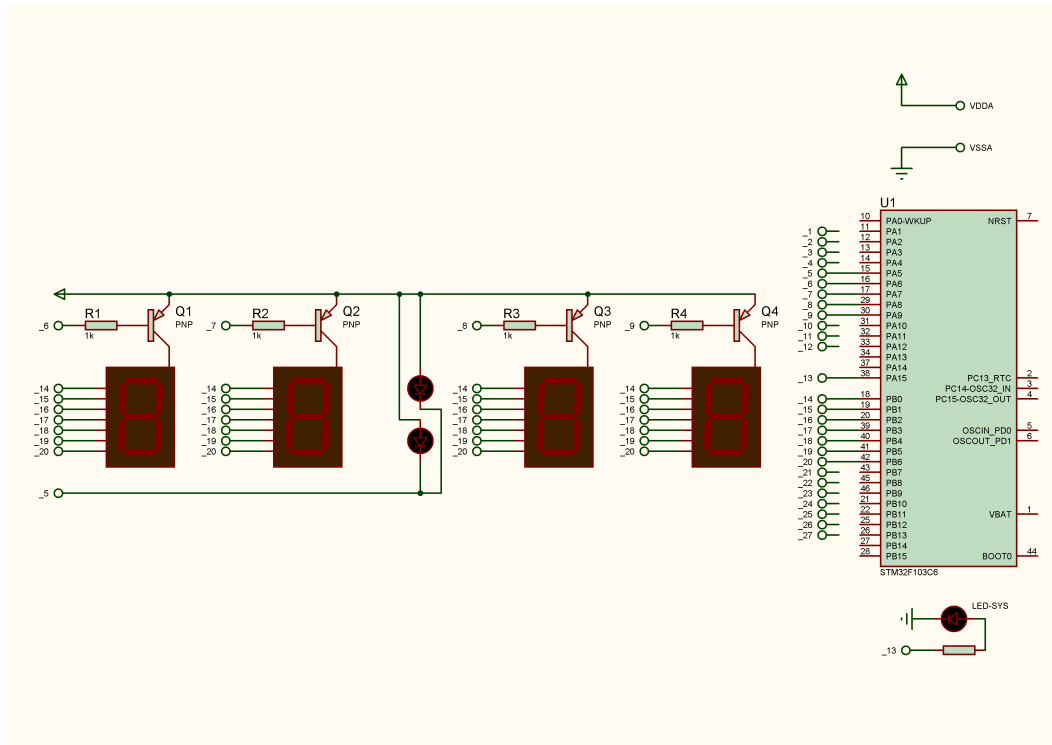
Hình 1.7: Simulation schematic in Proteus

Blink the two LEDs every second. Meanwhile, number 3 is displayed on the third seven segment LEDs and number 0 is displayed on the last one (to present 12 hour and

a half). The switching time for each seven segment LED is also a half of second (500ms). **Implement your code in the timer interrupt function.**

Report 1: Capture your schematic from Proteus and show in the report.

My schematic:



Hình 1.8: Simulation schematic for Lab2 Ex2 in Proteus.

Report 2: Present your source code in the **HAL_TIM_PeriodElapsedCallback** function.

My code:

```

1 int timerLED_SYS = 50;
2 int timer1 = 50;
3 int index_led = 0;
4 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
5 {
6     timerLED_SYS--;
7     timer1--;
8     if (timerLED_SYS <= 0){
9         timerLED_SYS = 50;
10        HAL_GPIO_TogglePin(LED_SYS_GPIO_Port, LED_SYS_Pin);
11        HAL_GPIO_TogglePin(PA5_GPIO_Port, PA5_Pin);
12    }
13    if (timer1 <= 0){
14        timer1 = 50;
15        if (index_led == 0){
16            display7SEG(1);
17            HAL_GPIO_WritePin(PA6_GPIO_Port, PA6_Pin,
18            GPIO_PIN_RESET);
19            HAL_GPIO_WritePin(PA7_GPIO_Port, PA7_Pin,
20            GPIO_PIN_SET);
21            HAL_GPIO_WritePin(PB8_GPIO_Port, PB8_Pin,
22            GPIO_PIN_SET);
23            HAL_GPIO_WritePin(PB9_GPIO_Port, PB9_Pin,
24            GPIO_PIN_SET);
25            index_led = 1;
26        }
27        else if (index_led == 1){
28            display7SEG(2);
29            HAL_GPIO_WritePin(PA6_GPIO_Port, PA6_Pin,
30            GPIO_PIN_SET);
31            HAL_GPIO_WritePin(PA7_GPIO_Port, PA7_Pin,
32            GPIO_PIN_RESET);
33            HAL_GPIO_WritePin(PB8_GPIO_Port, PB8_Pin,
34            GPIO_PIN_SET);
35            HAL_GPIO_WritePin(PB9_GPIO_Port, PB9_Pin,
36            GPIO_PIN_SET);
37            index_led = 2;
38        }
39        else if (index_led == 2){
40            display7SEG(3);
41            HAL_GPIO_WritePin(PA6_GPIO_Port, PA6_Pin,
42            GPIO_PIN_SET);
43            HAL_GPIO_WritePin(PA7_GPIO_Port, PA7_Pin,
44            GPIO_PIN_SET);
45            HAL_GPIO_WritePin(PB8_GPIO_Port, PB8_Pin,
46            GPIO_PIN_RESET);
47            HAL_GPIO_WritePin(PB9_GPIO_Port, PB9_Pin,
48            GPIO_PIN_SET);
49            index_led = 3;
50        }
51    }
52 }

```



```

37     }
38     else if (index_led == 3){
39         display7SEG(0);
40         HAL_GPIO_WritePin(PA6_GPIO_Port , PA6_Pin ,
GPIO_PIN_SET);
41         HAL_GPIO_WritePin(PA7_GPIO_Port , PA7_Pin ,
GPIO_PIN_SET);
42         HAL_GPIO_WritePin(PB8_GPIO_Port , PB8_Pin ,
GPIO_PIN_SET);
43         HAL_GPIO_WritePin(PB9_GPIO_Port , PB9_Pin ,
GPIO_PIN_RESET);
44         index_led = 0;
45     }
46 }
47 }

```

Program 1.5: My code for Lab2 Ex2

Short question: What is the frequency of the scanning process?

My answer: The frequency of the scanning process is 0.5Hz. As the switching time between 4 SEGMENTS is by 50 unit (= 500ms in this configuration), the period for scanning 4 SEGMENTS is 200 unit (= 2s). Therefore, the frequency is $1/2s = 0.5Hz$.

3.3 Exercise 3

Implement a function named **update7SEG(int index)**. An array of 4 integer numbers are declared in this case. The code skeleton in this exercise is presented as following:

```

1  const int MAX_LED = 4;
2  int index_led = 0;
3  int led_buffer[4] = {1, 2, 3, 4};
4  void update7SEG(int index){
5      switch (index){
6          case 0:
7              //Display the first 7SEG with led_buffer[0]
8              break;
9          case 1:
10             //Display the second 7SEG with led_buffer[1]
11             break;
12          case 2:
13             //Display the third 7SEG with led_buffer[2]
14             break;
15          case 3:
16             //Display the forth 7SEG with led_buffer[3]
17             break;
18          default:
19             break;
20      }
21 }

```

Program 1.6: An example for your source code

This function should be invoked in the timer interrupt, e.g `update7SEG(index_led++)`. The variable **index_led** is updated to stay in a valid range, which is from 0 to 3.

Report 1: Present the source code of the `update7SEG` function.

My code:

```

1  void update7SEG(int index){
2      switch (index){
3          case 0:
4              display7SEG(led_buffer[0]);
5              HAL_GPIO_WritePin(PA6_GPIO_Port, PA6_Pin,
6              GPIO_PIN_RESET);
7              HAL_GPIO_WritePin(PA7_GPIO_Port, PA7_Pin,
8              GPIO_PIN_SET);
9              HAL_GPIO_WritePin(PB8_GPIO_Port, PB8_Pin,
10             GPIO_PIN_SET);
11             HAL_GPIO_WritePin(PB9_GPIO_Port, PB9_Pin,
12             GPIO_PIN_SET);
13             break;
14          case 1:
15             display7SEG(led_buffer[1]);
16             HAL_GPIO_WritePin(PA6_GPIO_Port, PA6_Pin,
17             GPIO_PIN_SET);
18             HAL_GPIO_WritePin(PA7_GPIO_Port, PA7_Pin,
19             GPIO_PIN_RESET);

```

```

14         HAL_GPIO_WritePin(PB8_GPIO_Port , PB8_Pin ,
GPIO_PIN_SET);
15         HAL_GPIO_WritePin(PB9_GPIO_Port , PB9_Pin ,
GPIO_PIN_SET);
16         break;
17     case 2:
18         display7SEG(led_buffer[2]);
19         HAL_GPIO_WritePin(PA6_GPIO_Port , PA6_Pin ,
GPIO_PIN_SET);
20         HAL_GPIO_WritePin(PA7_GPIO_Port , PA7_Pin ,
GPIO_PIN_SET);
21         HAL_GPIO_WritePin(PB8_GPIO_Port , PB8_Pin ,
GPIO_PIN_RESET);
22         HAL_GPIO_WritePin(PB9_GPIO_Port , PB9_Pin ,
GPIO_PIN_SET);
23         break;
24     case 3:
25         display7SEG(led_buffer[3]);
26         HAL_GPIO_WritePin(PA6_GPIO_Port , PA6_Pin ,
GPIO_PIN_SET);
27         HAL_GPIO_WritePin(PA7_GPIO_Port , PA7_Pin ,
GPIO_PIN_SET);
28         HAL_GPIO_WritePin(PB8_GPIO_Port , PB8_Pin ,
GPIO_PIN_SET);
29         HAL_GPIO_WritePin(PB9_GPIO_Port , PB9_Pin ,
GPIO_PIN_RESET);
30         break;
31     default:
32         break;
33 }
34 }

```

Program 1.7: My code for update7SEG function

Report 2: Present the source code in the **HAL_TIM_PeriodElapsedCallback** function.

My code:

```

1  int timerLED_SYS = 50;
2  int timer1 = 50;
3  int index_led = 0;
4  const int MAX_LED = 4;
5  int led_buffer[4] = {1, 2, 3, 4};
6  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
    {
7      timerLED_SYS--;
8      timer1--;
9      if (timerLED_SYS <= 0){
10         timerLED_SYS = 50;

```

```

11     HAL_GPIO_TogglePin(LED_SYS_GPIO_Port , LED_SYS_Pin);
12     HAL_GPIO_TogglePin(PA5_GPIO_Port , PA5_Pin);
13 }
14 if (timer1 <= 0){
15     timer1 = 50;
16     update7SEG(index_led++);
17     if (index_led >= MAX_LED) index_led = 0;
18 }
19 }

```

Program 1.8: My code for Lab2 Ex3

Students are proposed to change the values in the **led_buffer** array for unit test this function, which is used afterward.

3.4 Exercise 4

Change the period of invoking update7SEG function in order to set the frequency of 4 seven segment LEDs to 1Hz. The DOT is still blinking every second.

Report 1: Present the source code in the **HAL_TIM_PeriodElapsedCallback**.
My code:

```

1 int timerLED_SYS = 50;
2 int timer1 = 25; //25 unit -> 250ms -> 4*250ms = 1s -> 1Hz
3 int index_led = 0;
4 const int MAX_LED = 4;
5 int led_buffer[4] = {1, 2, 3, 4};
6 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
7 {
8     timerLED_SYS--;
9     timer1--;
10    if (timerLED_SYS <= 0){
11        timerLED_SYS = 50;
12        HAL_GPIO_TogglePin(LED_SYS_GPIO_Port , LED_SYS_Pin);
13        HAL_GPIO_TogglePin(PA5_GPIO_Port , PA5_Pin);
14    }
15    if (timer1 <= 0){
16        timer1 = 25;
17        update7SEG(index_led++);
18        if (index_led >= MAX_LED) index_led = 0;
19    }
20 }

```

Program 1.9: My code for Lab2 Ex3

3.5 Exercise 5

Implement a digital clock with **hour** and **minute** information displayed by 2 seven segment LEDs. The code skeleton in the **main** function is presented as follows:

```
1 int hour = 15, minute = 8, second = 50;
2
3 while(1){
4     second++;
5     if (second >= 60){
6         second = 0;
7         minute++;
8     }
9     if(minute >= 60){
10        minute = 0;
11        hour++;
12    }
13    if(hour >=24){
14        hour = 0;
15    }
16    updateClockBuffer();
17    HAL_Delay(1000);
18 }
```

Program 1.10: An example for your source code

The function **updateClockBuffer** will generate values for the array **led_buffer** according to the values of hour and minute. In the case these values are 1 digit number, digit 0 is added.

Report 1: Present the source code in the **updateClockBuffer** function.

My code:

```
1 void updateClockBuffer() {
2     // Extract individual digits from hour and minute
3     led_buffer[0] = hour / 10;           // Hours tens digit
4     led_buffer[1] = hour % 10;           // Hours units digit
5     led_buffer[2] = minute / 10;         // Minutes tens digit
6     led_buffer[3] = minute % 10;         // Minutes units digit
7 }
```

Program 1.11: My code for updateClockBuffer function

3.6 Exercise 6

The main target from this exercise to reduce the complexity (or reduce code processing) in the timer interrupt. The time consumed in the interrupt can lead to the nested interrupt issue, which can crash the whole system. A simple solution can disable the timer whenever the interrupt occurs, the enable it again. However, the real-time processing is not guaranteed anymore.

In this exercise, a software timer is created and its counter is count down every timer interrupt is raised (every 10ms). By using this timer, the **Hal_Delay(1000)** in the main function is removed. In a MCU system, non-blocking delay is better than blocking delay. The details to create a software timer are presented bellow. The source code is added to your current program, **do not delete the source code you have on Exercise 5.**

Step 1: Declare variables and functions for a software timer, as following:

```
1  /* USER CODE BEGIN 0 */
2  int timer0_counter = 0;
3  int timer0_flag = 0;
4  int TIMER_CYCLE = 10;
5  void setTimer0(int duration){
6      timer0_counter = duration /TIMER_CYCLE;
7      timer0_flag = 0;
8  }
9  void timer_run(){
10     if(timer0_counter > 0){
11         timer0_counter--;
12         if(timer0_counter == 0) timer0_flag = 1;
13     }
14 }
15 /* USER CODE END 0 */
```

Program 1.12: Software timer based timer interrupt

Please change the **TIMER_CYCLE** to your timer interrupt period. In the manual code above, it is **10ms**.

Step 2: The **timer_run()** is invoked in the timer interrupt as following:

```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     timer_run();
4
5     //YOUR OTHER CODE
6 }
```

Program 1.13: Software timer based timer interrupt

Step 3: Use the timer in the main function by invoked setTimer0 function, then

check for its flag (timer0_flag). An example to blink an LED connected to PA5 using software timer is shown as follows:

```
1 setTimer0(1000);  
2 while (1){  
3     if(timer0_flag == 1){  
4         HAL_GPIO_TogglePin(LED_RED_GPIO_Port , LED_RED_Pin);  
5         setTimer0(2000);  
6     }  
7 }
```

Program 1.14: Software timer is used in main function to blink the LED

Report 1: if in line 1 of the code above is miss, what happens after that and why?

My answer: If line 1 is missing, then timer0_flag will always be 0. And because of that, the if condition in the timer_run() function (timer0_counter > 0) will never be satisfied (timer0_counter is initialized as 0).

As a result, the timer0_flag will never be set to 1, and the condition in the while loop (timer0_flag == 1) will never be true. Therefore, the while loop will never execute the code inside it.

So, nothing happens.

Report 2: if in line 1 of the code above is changed to setTimer0(1), what happens after that and why?

My answer: If line 1 is changed to setTimer0(1), then timer0_counter will be set to 0 ($1 / 10 = 0.1$, which is truncated to 0).

And because of that, the if condition in the timer_run() function (timer0_counter > 0) will never be satisfied, then timer0_flag will never be set to 1.

Hence, this will lead to the same situation as in Report 1, where the while loop will never execute the code inside it.

Report 3: if in line 1 of the code above is changed to setTimer0(10), what is changed compared to 2 first questions and why?

My answer: If line 1 is changed to setTimer0(10), then timer0_counter will be set to 1 ($10 / 10 = 1$).

And because of that, the if condition in the timer_run() function (timer0_counter > 0) will be satisfied once, then timer0_counter will be decremented by 1 and timer0_flag will be set to 1 (satisfied the condition timer0_counter == 0).

Hence, the while loop will execute the code inside it once, toggling the LED_RED pin.

After that, timer0_counter will be set to 200 ($2000 / 10 = 200$) and the process will repeat.

So, the LED_RED pin will toggle every 2 seconds -> full cycle is 4 seconds -> frequency is 0.25Hz.

What is changed compared to the 2 first questions: In the first two questions,

the while loop never executed the code inside it, so nothing happened. But in this case, the while loop executes the code inside it once, toggling the LED_RED pin.

3.7 Exercise 7

Upgrade the source code in Exercise 5 (update values for hour, minute and second) by using the software timer and remove the HAL_Delay function at the end. Moreover, the DOT (connected to PA4) of the digital clock is also moved to main function.

Report 1: Present your source code in the while loop on main function.

My code:

```
1 int hour = 15, minute = 8, second = 50;
2 int led_buffer[4] = {1, 2, 3, 4};
3 setTimerLED_SYS(500);
4 setTimer0(1000);
5 while (1){
6     if (timerLED_SYS_flag == 1){
7         HAL_GPIO_TogglePin(LED_SYS_GPIO_Port, LED_SYS_Pin);
8         HAL_GPIO_TogglePin(PA5_GPIO_Port, PA5_Pin);
9         setTimerLED_SYS(500);
10    }
11    if (timer0_flag == 1){
12        second++;
13        if (second >= 60){
14            second = 0;
15            minute++;
16        }
17        if (minute >= 60){
18            minute = 0;
19            hour++;
20        }
21        if (hour >= 24){
22            hour = 0;
23        }
24        updateClockBuffer();
25        setTimer0(1000);
26    }
27 }
```

Program 1.15: My code for Lab2 Ex7

3.8 Exercise 8

Move also the update7SEG() function from the interrupt timer to the main. Finally, the timer interrupt only used to handle software timers. All processing (or complex

computations) is move to an infinite loop on the main function, optimizing the complexity of the interrupt handler function.

Report 1: Present your source code in the the main function. In the case more extra functions are used (e.g. the second software timer), present them in the report as well.

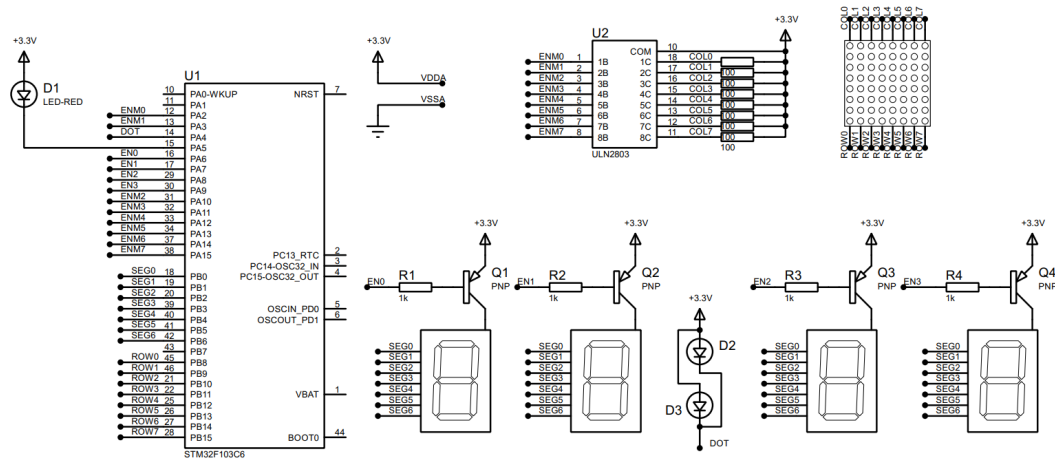
My code:

```
1 int hour = 15, minute = 8, second = 50;
2 int index_led = 0;
3 const int MAX_LED = 4;
4 int led_buffer[4] = {1, 2, 3, 4};
5 setTimerLED_SYS(500);
6 setTimer0(1000);
7 setTimer1(250);
8 while (1){
9     if (timerLED_SYS_flag == 1){
10         HAL_GPIO_TogglePin(LED_SYS_GPIO_Port, LED_SYS_Pin);
11         HAL_GPIO_TogglePin(PA5_GPIO_Port, PA5_Pin);
12         setTimerLED_SYS(500);
13     }
14     if (timer0_flag == 1){
15         second++;
16         if (second >= 60){
17             second = 0;
18             minute++;
19         }
20         if (minute >= 60){
21             minute = 0;
22             hour++;
23         }
24         if (hour >= 24){
25             hour = 0;
26         }
27         updateClockBuffer();
28         setTimer0(1000);
29     }
30     if (timer1_flag == 1){
31         update7SEG(index_led++);
32         if (index_led >= MAX_LED) index_led = 0;
33         setTimer1(250);
34     }
35 }
```

Program 1.16: My code for Lab2 Ex7

3.9 Exercise 9

This is an extra works for this lab. A LED Matrix is added to the system. A reference design is shown in figure bellow:

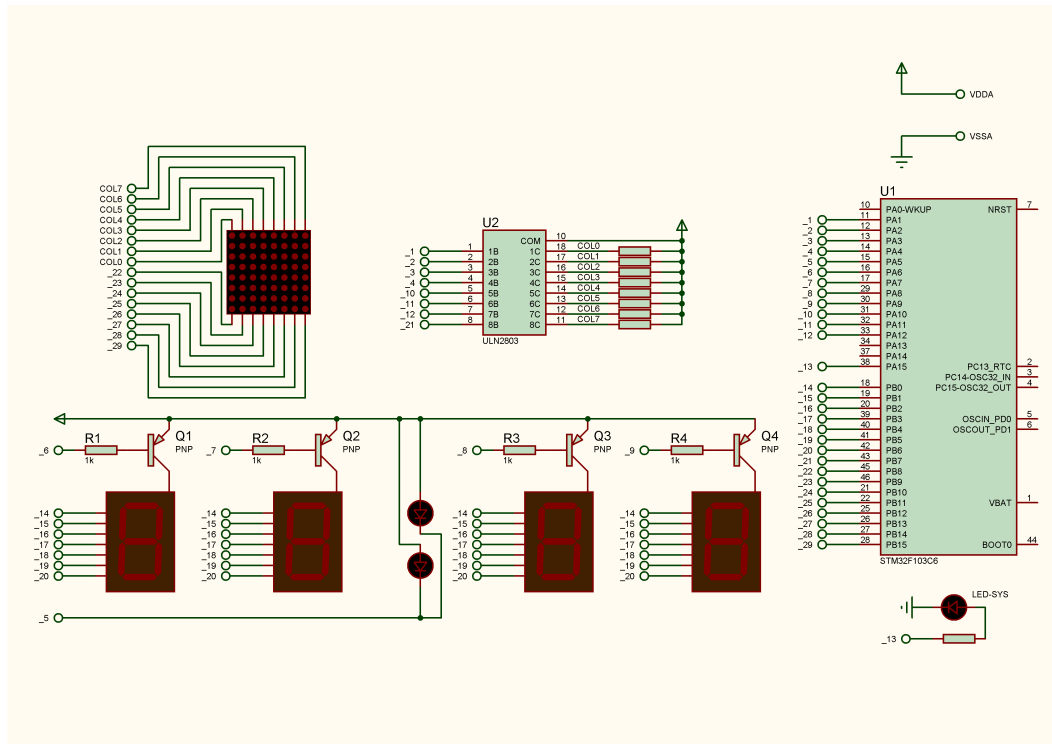


Hình 1.9: LED matrix is added to the simulation

In this schematic, two new components are added, including the **MATRIX-8X8-RED** and **ULN2803**, which is an NPN transistor array to enable the power supply for a column of the LED matrix. Students can change the enable signal (from ENM0 to ENM7) if needed. Finally, the data signal (from ROW0 to ROW7) is connected to PB8 to PB15.

Report 1: Present the schematic of your system by capturing the screen in Proteus.

My schematic:



Hình 1.10: Simulation schematic for Lab2 Ex9 in Proteus.

Report 2: Implement the function, `updateLEDMatrix(int index)`, which is similarly to 4 seven led segments.

My code:

```
1 const int MAX_LED_MATRIX = 8;
2 int index_led_matrix = 0;
3 uint8_t matrix_buffer[8] = {0x00, 0xFC, 0x12, 0x11, 0x11, 0
  x12, 0xFC, 0x00}; //Character "A"
4 void setRowData(uint8_t data){
5     HAL_GPIO_WritePin(PB8_GPIO_Port, PB8_Pin, !((data >> 0)
6     & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
7     HAL_GPIO_WritePin(PB9_GPIO_Port, PB9_Pin, !((data >> 1)
8     & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
9     HAL_GPIO_WritePin(PB10_GPIO_Port, PB10_Pin, !((data >>
10    2) & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
11    HAL_GPIO_WritePin(PB11_GPIO_Port, PB11_Pin, !((data >>
12    3) & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
13    HAL_GPIO_WritePin(PB12_GPIO_Port, PB12_Pin, !((data >>
14    4) & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
15    HAL_GPIO_WritePin(PB13_GPIO_Port, PB13_Pin, !((data >>
16    5) & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
17    HAL_GPIO_WritePin(PB14_GPIO_Port, PB14_Pin, !((data >>
18    6) & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
```

```

12     HAL_GPIO_WritePin(PB15_GPIO_Port, PB15_Pin, !((data >>
13     7) & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
14 }
15 void updateLEDMatrix(int index){
16     switch (index){
17         case 0:
18             HAL_GPIO_WritePin(PA1_GPIO_Port, PA1_Pin,
19             GPIO_PIN_RESET);
20             setRowData(matrix_buffer[index]);
21             break;
22         case 1:
23             HAL_GPIO_WritePin(PA2_GPIO_Port, PA2_Pin,
24             GPIO_PIN_RESET);
25             setRowData(matrix_buffer[index]);
26             break;
27         case 2:
28             HAL_GPIO_WritePin(PA3_GPIO_Port, PA3_Pin,
29             GPIO_PIN_RESET);
30             setRowData(matrix_buffer[index]);
31             break;
32         case 3:
33             HAL_GPIO_WritePin(PA4_GPIO_Port, PA4_Pin,
34             GPIO_PIN_RESET);
35             setRowData(matrix_buffer[index]);
36             break;
37         case 4:
38             HAL_GPIO_WritePin(PA10_GPIO_Port, PA10_Pin,
39             GPIO_PIN_RESET);
40             setRowData(matrix_buffer[index]);
41             break;
42         case 5:
43             HAL_GPIO_WritePin(PA11_GPIO_Port, PA11_Pin,
44             GPIO_PIN_RESET);
45             setRowData(matrix_buffer[index]);
46             break;
47         case 6:
48             HAL_GPIO_WritePin(PA12_GPIO_Port, PA12_Pin,
49             GPIO_PIN_RESET);
50             setRowData(matrix_buffer[index]);
51             break;
52         case 7:
53             HAL_GPIO_WritePin(PB7_GPIO_Port, PB7_Pin,
54             GPIO_PIN_RESET);
55             setRowData(matrix_buffer[index]);
56             break;
57         default:
58             break;
59     }

```

51 }

Program 1.17: Function to display data on LED Matrix

Students are free to choose the invoking frequency of this function. However, this function is supposed to be invoked in the main function. Finally, please update the **matrix_buffer** to display character "A".

3.10 Exercise 10

Create an animation on LED matrix, for example, the character is shifted to the left.

Report 1: Briefly describe your solution and present your source code in the report.

My solution: To animate the LED matrix, I created a function called `shiftMatrixBufferLeft` that shifts the contents of the `matrix_buffer` array UPWARD by one position. The first element is moved to the end of the array, creating a continuous scrolling effect.

Pros: This approach is simple and effective.

Cons: The animation used a lot of CPU resources to shift 8 bits at high frequency, which might overload the system.

```
1 void shiftMatrixBufferLeft() {
2     unsigned char temp = matrix_buffer[0];
3
4     for (int i = 0; i <= MAX_LED_MATRIX - 2; i++) {
5         matrix_buffer[i] = matrix_buffer[i + 1];
6     }
7
8     matrix_buffer[MAX_LED_MATRIX - 1] = temp;
9 }
```

Program 1.18: Function to shift the matrix buffer to the left

My final code: There are some custom functions used in the final exercise.

```
1 void togglePin(PINMAP pin, bool state) {
2     //safety check
3     if (pin.port == NULL) {
4         return;
5     }
6
7     HAL_GPIO_WritePin(pin.port, pin.pin, state ?
8     GPIO_PIN_SET : GPIO_PIN_RESET);
9 }
10 void toggleSeg(PINMAP segment[7], uint8_t map[12], uint8_t
11     number) {
12     //safety check
13     if (number < 0 || number > 10) {
```

```

13     number = 11; //E - ERROR
14 }
15
16 for (int i = 0; i < 7; i++) {
17     uint8_t state = (map[number] >> (6 - i)) & 0x01;
18     togglePin(segment[i], state);
19 }
20 }
21
22 void blinkPin(PINMAP pin) {
23     HAL_GPIO_TogglePin(pin.port, pin.pin);
24 }
25
26 void blinkLED_SYS() {
27     HAL_GPIO_TogglePin(LED_SYS.port, LED_SYS.pin);
28 }
29
30 void blinkLED_SYS_TIM(int duration) {
31     if (timerLED_SYS_flag == 1) {
32         setTimerLED_SYS(duration);
33         blinkLED_SYS();
34     }
35 }

```

Program 1.19: My custom functions

```

1 // LED Matrix variables
2 const int MAX_LED_MATRIX = 8;
3 int index_led_matrix = 0;
4 unsigned char matrix_buffer[8] = {
5     //ROWx
6     //x=
7     //76543210
8     // Char "A"
9     0x00,
10    0xFC,
11    0x12,
12    0x11,
13    0x11,
14    0x12,
15    0xFC,
16    0x00
17 };
18
19 // Clock variables
20 const int MAX_LED = 4;
21 int index_led = 0;
22 int led_buffer[4] = {1, 2, 3, 4};
23 int hour = 23;
24 int minute = 58;

```

```

25 int second = 58;
26
27 void update7SEG(int index){
28     togglePin(PA6, 1);
29     togglePin(PA7, 1);
30     togglePin(PA8, 1);
31     togglePin(PA9, 1);
32
33     int digitToShow = led_buffer[index];
34
35     toggleSeg(segMent_1, segMapAnode, digitToShow);
36
37     switch(index) {
38         case 0:
39             togglePin(PA6, 0);
40             break;
41         case 1:
42             togglePin(PA7, 0);
43             break;
44         case 2:
45             togglePin(PA8, 0);
46             break;
47         case 3:
48             togglePin(PA9, 0);
49             break;
50         default:
51             togglePin(PA6, 1);
52             togglePin(PA7, 1);
53             togglePin(PA8, 1);
54             togglePin(PA9, 1);
55             break;
56     }
57 }
58
59 void updateClockBuffer() {
60     // Extract individual digits from hour and minute
61     led_buffer[0] = hour / 10;           // Hours tens digit
62     led_buffer[1] = hour % 10;           // Hours units digit
63     led_buffer[2] = minute / 10;         // Minutes tens digit
64     led_buffer[3] = minute % 10;         // Minutes units
65     digit
66
67     // Handle time increment (increment every second)
68     second++;
69     if (second >= 60) {
70         second = 0;
71         minute++;
72         if (minute >= 60) {
73             minute = 0;

```

```

73         hour++;
74         if (hour >= 24) {
75             hour = 0;
76         }
77     }
78 }
79 }
80
81 // LED Matrix Functions
82 void setRowData(unsigned char row_data) {
83     // Set ROW0-ROW7 - LOW ACTIVE
84     togglePin(PB8, !((row_data >> 0) & 0x01)); // ROW0
85     togglePin(PB9, !((row_data >> 1) & 0x01)); // ROW1
86     togglePin(PB10, !((row_data >> 2) & 0x01)); // ROW2
87     togglePin(PB11, !((row_data >> 3) & 0x01)); // ROW3
88     togglePin(PB12, !((row_data >> 4) & 0x01)); // ROW4
89     togglePin(PB13, !((row_data >> 5) & 0x01)); // ROW5
90     togglePin(PB14, !((row_data >> 6) & 0x01)); // ROW6
91     togglePin(PB15, !((row_data >> 7) & 0x01)); // ROW7
92 }
93
94 void setColumnEnable(int column, int enable) {
95     // Set ENM0-ENM7 - HIGH ACTIVE
96     switch(column) {
97         case 0: togglePin(PA1, !enable); break; // ENM0
98         case 1: togglePin(PA2, !enable); break; // ENM1
99         case 2: togglePin(PA3, !enable); break; // ENM2
100        case 3: togglePin(PA4, !enable); break; // ENM3
101        case 4: togglePin(PA10, !enable); break; // ENM4
102        case 5: togglePin(PA11, !enable); break; // ENM5
103        case 6: togglePin(PA12, !enable); break; // ENM6
104        case 7: togglePin(PB7, !enable); break; // ENM7
105    }
106 }
107
108 void updateLEDMatrix(int index) {
109     // Turn off all columns first
110     for(int col = 0; col < 8; col++) {
111         setColumnEnable(col, 0);
112     }
113
114     // Set row data and enable the current column
115     if(index < MAX_LED_MATRIX) {
116         setRowData(matrix_buffer[index]);
117         setColumnEnable(index, 1);
118     }
119 }
120
121 void shiftMatrixBufferLeft() {

```



```

122     unsigned char temp = matrix_buffer[0];
123
124     for (int i = 0; i <= MAX_LED_MATRIX - 2; i++) {
125         matrix_buffer[i] = matrix_buffer[i + 1];
126     }
127
128     matrix_buffer[MAX_LED_MATRIX - 1] = temp;
129 }

```

Program 1.20: My functions

```

1 //LED_SYS SETUP
2 int durLED_SYS = 50;
3 setTimerLED_SYS(durLED_SYS);
4 //LED_SYS SETUP
5
6 //DIGITAL CLOCK SETUP (Restore 7-segment functionality)
7 int multiplexTimer = 5; // 50ms per display (5 * 10ms = 50
   ms) (speed up 5x for better visual)
8 int colonTimer = 50; // 0.5 second for DOT blinking (50
   * 10ms = 500ms)
9 int clockTimer = 10; // 100ms for clock update (10 * 10
   ms = 100ms) (speed up 10x for demo)
10 int currentDisplay = 0;
11
12 // Initialize clock buffer with current time
13 updateClockBuffer();
14 setTimer1(multiplexTimer); // Restore 7-segment
   multiplexing
15 setTimer2(colonTimer);
16 setTimer3(clockTimer);
17 //DIGITAL CLOCK SETUP
18
19 //LED MATRIX SETUP
20 int matrixTimer = 1; // 10ms for matrix display (1 *
   10ms = 10ms)
21 int animationTimer = 10; // 100ms for animation shift (10 *
   10ms = 100ms)
22 setTimer4(matrixTimer);
23 setTimer5(animationTimer);
24 //LED MATRIX SETUP
25 while (1)
26 {
27     //LED_SYS OPERATION
28     blinkLED_SYS_TIM(durLED_SYS);
29     //LED_SYS OPERATION
30
31     //DIGITAL CLOCK OPERATION
32     if (timer1_flag == 1)
33     {

```

```

34     setTimer1(multiplexTimer);
35     update7SEG(currentDisplay);
36     currentDisplay = (currentDisplay + 1) % 4;
37 }
38
39 if (timer2_flag == 1)
40 {
41     setTimer2(colonTimer);
42     blinkPin(PA5);
43 }
44
45 if (timer3_flag == 1)
46 {
47     setTimer3(clockTimer);
48     updateClockBuffer();
49 }
50 //DIGITAL CLOCK OPERATION
51
52 //LED MATRIX OPERATION
53 if (timer4_flag == 1)
54 {
55     setTimer4(matrixTimer);
56     updateLEDMatrix(index_led_matrix);
57     index_led_matrix = (index_led_matrix + 1) %
MAX_LED_MATRIX;
58 }
59
60 //LED MATRIX ANIMATION
61 if (timer5_flag == 1)
62 {
63     setTimer5(animationTimer);
64     shiftMatrixBufferLeft();
65 }
66 //LED MATRIX ANIMATION
67 }

```

Program 1.21: My main loop